Sparse Koopman Autoencoders Identify Local Dynamical Regimes in Multibasin Systems

Anonymous Author(s)
Affiliation
Address
`email`

Abstract

Koopman autoencoders (KAEs) seek a higher-dimensional latent representation in which nonlinear dynamics evolve linearly. However, many interesting systems have multiple basins of attraction, and both theoretical and empirical work has shown these multibasin systems cannot generally admit a single finite-dimensional global Koopman embedding under standard assumptions. We posit that encoders with a sparsity-inducing objective encouraging few active latent coefficients will provide latent supports as an inspectable basin-modeling principle for Koopman autoencoders. We use these encoders producing sparse latents in training Sparse Koopman Autoencoders (SKAEs) without basin labels or other regime annotations, and treat the learned latent supports as model-produced regime variables after training. Across a range of procedurally generated multibasin systems and chaotic flows, SKAEs have superior forecasting performance compared to dense-latent KAEs. The latent supports produced by SKAEs are both essential for the quality of the representation and useful for identifying basins on held-out basin interior states, whereas dense-latent KAEs collapse to an uninformative single family. These results identify sparse latents and their corresponding supports as label-free, interpretable regime variables for Koopman learning in nonlinear systems with multiple local dynamical laws.

1 Introduction

Learning representations for nonlinear dynamical systems remains a central problem in machine learning, scientific computing, and control. A particularly useful goal is to embed nonlinear dynamics into coordinates in which the evolution is linear, or at least locally linear, because linear models can be combined with mature tools for prediction, estimation, and control, including model predictive control and linear quadratic regulation [27, 15, 20, 17, 26]. This goal is also well motivated theoretically. Classical dynamical-systems results provide local topological linearizations near hyperbolic fixed points [14, 16], while Koopman eigenfunction constructions can yield linearizing coordinates on basins of attraction for stable equilibria and periodic orbits [22, 21]. Thus, nonlinear systems may admit linear dynamics with a specific change of coordinates.

The difficulty is that these results are primarily existential. They establish when useful coordinates can exist, but they do not by themselves provide a finite-data procedure for constructing such coordinates. Koopman operator theory formalizes the linearization idea by lifting nonlinear state evolution to a linear operator acting on observables [18, 19, 28, 6]. This viewpoint has inspired data-driven approximations of Koopman dynamics, such as DMD and eDMD [32, 35, 36], and more recently to Koopman autoencoders, which learn an encoder from state space to a latent representation, a linear latent transition matrix, and a decoder back to state space [34, 25, 30, 3, 33, 10]. These models provide an appealing route from existence results to practical learning: rather than hand-designing observables, they attempt to learn the embedding directly from trajectory data.

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

A central challenge for Koopman approximations is that many practically interesting nonlinear systems typically contain multiple equilibria or basins of attraction; for example, a damped mechanical system can settle into different wells, a biological or chemical model can approach different stable equilibria, and a controlled system can move between regions with distinct local dynamics. However, theory shows that these systems cannot generally admit a single finite-dimensional global linearization with a continuous encoder-decoder pair [22, 5, 31, 24], and empirical evidence supports this in practice [10]. In such multistable systems, it is possible to linearize the dynamics within basins, but the linearizations needed in different basins of attraction are generally incompatible with each other. Therefore, for multi-attractor systems, the right objective is not to force all basins through one continuous global linearization, but to discover and use distinct local linearizations.

This paper proposes inducing sparse latent structure as a practical and interpretable method for Koopman learning of multi-attractor systems. By inducing the state to activate only a small subset of coordinates in lifted latent space, we can incentivize different regions of state to occupy different active coordinates, since nearby states should have similar latent embeddings, and thus enable local linearizations. In this sparse representation, nonzero coefficient values in a latent vector can provide continuous coordinates for prediction within a selected region, while the active indices provide a discrete support object that can be inspected across states and used to select local linearizations.

Sparse coding and LISTA-style encoders [13, 8, 1] provide the right inductive bias for this purpose. They naturally induce piecewise-linear, union-of-subspaces structure [29, 7, 9, 13], which is well-matched to this setting where different attractors or basins require different local linearizations in theory. We hypothesize that sparse-latent Koopman autoencoders will assign local dynamical regimes to distinct latent support families, yielding a learned regime variable without basin supervision. Then, periodically decoding and re-encoding the state can refresh the quality of the latent embedding over time by re-selecting the correct local dynamics when trajectories move across the state space [10].

Contributions. We introduce Sparse Koopman Autoencoders (SKAEs), showing that sparse latent supports can serve as label-free regime variables that reconcile basin-specific Koopman linearizations with the practicality of a single trained model. Across multibasin and chaotic benchmarks, SKAEs (1) improve long-horizon forecasting while (2) producing supports that are functionally necessary for accurate rollouts and (3) interpretable as emergent local dynamical regimes.

2 Problem formulation

The introduction motivates a multibasin version of Koopman learning. Given sampled trajectories of a nonlinear system, the aim is to learn a state-reconstructing latent representation whose evolution is linear, without being told which basin or local dynamical regime generated each observation. We formulate the problem at the cadence of the stored benchmark observations. For each system, let $\mathcal{X} \subseteq \mathbb{R}^{d_x}$ be the observed state space and let

$$x_{k+1} = F_{\Delta t}(x_k), \qquad x_k \in \mathcal{X}, \tag{1}$$

where $x_k \in \mathbb{R}^{d_x}$ is the state at the kth stored sample, $F_{\Delta t}$ denotes the transformation to the next discrete observation step based on underlying dynamics, and Δt is the system-specific observation interval. Forecasting is therefore defined by repeatedly applying the map $F_{\Delta t}$; a horizon of h means h applications of $F_{\Delta t}$; we denote k compositions as $F_{\Delta t}^{(k)}$. Because Δt may differ across systems, equal values of h may yield different physical time.

Basins of attraction. Many systems of interest are multistable: different initial conditions can approach different long-run behaviors. A fixed point is a state x^* satisfying $F_{\Delta t}(x^*) = x^*$. More generally, a trajectory may approach an attractor A, such as a stable equilibrium, limit cycle, countable finite set, or chaotic invariant set. The basin of attraction of A is the set of initial conditions whose trajectories converge to A:

$$\mathcal{B}(A) = \left\{ x_0 \in \mathbb{R}^{d_x} : \inf_{a \in A} \left\| F_{\Delta t}^{(k)}(x_0) - a \right\|_2 \to 0 \text{ as } k \to \infty \right\}. \tag{2}$$

When multiple attractors coexist, their basins partition the state space up to basin boundaries. This defines the setup for this paper. The task is to learn a representation that can support linear prediction where the appropriate linear law depends on where the state lies in the multibasin geometry.

figs/fig_benchmark_support_dysts_composite.pdf

Figure 1: Top: procedurally generated multibasin systems where gray streamlines are the true vector field, colored dots mark support-family/basin matches, and black dots mark support-family/basin mismatches. The learned LISTA support families are mapped post hoc to each family's dominant evaluation basin. Bottom: Dysts phase portraits with held-out truth in gray and forecasts in color. Red lines are predictions by the seed-0 dense-latent MLP, and blue lines are from the best seed-0 LISTA variant. The dense MLP makes significant mistakes in forecasting the Chua system, unlike LISTA.

Koopman theory. The stored-step map $F_{\Delta t}$ induces a Koopman operator $\mathcal{K}$ on scalar observables $g : \mathcal{X} \to \mathbb{R}$ by composition, $(\mathcal{K}g)(x) = g(F_{\Delta t}(x))$. Although $F_{\Delta t}$ may be nonlinear, $\mathcal{K}$ is linear on the space of observables. A finite-dimensional Koopman approximation searches for a vector observable $\Phi : \mathcal{X} \to \mathbb{R}^{d_z}$ and a matrix K such that $\Phi(F_{\Delta t}(x)) \approx K\Phi(x)$, while retaining enough information to reconstruct the state.

Koopman autoencoders. We study Koopman autoencoders (KAEs) in this setting. The model learns an encoder $E_\theta : \mathbb{R}^{d_x} \to \mathbb{R}^{d_z}$, a decoder $D_\phi : \mathbb{R}^{d_z} \to \mathbb{R}^{d_x}$, and a latent linear transition matrix $K \in \mathbb{R}^{d_z \times d_z}$. For observation x_k at arbitrary timestep k, the corresponding open-loop h-step forecast is

$$\hat{x}_{k+h} = D_\phi\big(K^h E_\theta(x_k)\big) . \tag{3}$$

The same matrix K is applied at every step of the rollout, so any state-dependent structure required for prediction must be encoded in the learned representation rather than in a supplied regime label.

Training windows. Training uses windows of adjacent stored observations. For a batch of B windows of rollout length L, $\{x_{b,k}, x_{b,k+1}, \ldots, x_{b,k+L}\}_{b=1}^{B}$, where b indexes windows and ℓ indexes offsets within a window, the model encodes each observed state, rolls out from the initial encoded state, and decodes both reconstructed observations and latent forecasts. The encoded/reconstructed quantities are defined for $\ell = 0, \ldots, L$, while rollout/forecast quantities are defined for $\ell = 1, \ldots, L$:

$$z^{\mathrm{enc}}_{b,k+\ell} = E_\theta(x_{b,k+\ell}), \quad z^{\mathrm{roll}}_{b,k+\ell} = K^\ell z^{\mathrm{enc}}_{b,k}, \quad x^{\mathrm{rec}}_{b,k+\ell} = D_\phi(z^{\mathrm{enc}}_{b,k+\ell}), \quad \hat{x}_{b,k+\ell} = D_\phi(z^{\mathrm{roll}}_{b,k+\ell}). \tag{4}$$

The training losses described later are built from these reconstructed and rolled-out quantities. Each application of K is interpreted as one stored benchmark step, matching the observation map in eq. (1).

Model discovers basins unsupervised. In the intended training and deployment setting, the model is not given basin labels, the number of basins, or trajectory-to-basin assignments. When benchmark basin labels are available, they are reserved for post-hoc evaluation; they are not used to choose the model, train the encoder or transition, or select supports for support-conditioned predictions.

3 Method

3.1 Sparse supports as local regime variables

A finite-dimensional Koopman representation for the multibasin systems in Section 2 should encode two complementary quantities: (i) continuous coordinates that linearize the dynamics within a basin, and (ii) indicate which local linearization is active. We use sparsity to couple these two roles. If the state x is represented by a sparse latent code z, then the support of z can act as a regime variable, while the nonzero coefficient values provide coordinates for the corresponding local linear representation.

Given an overcomplete dictionary D, sparse coding estimates z by solving the Lasso objective

$$z^{\star}(x) = \arg\min_{z \in \mathbb{R}^{d_z}} \frac{1}{2} \left\| x - Dz \right\|_2^2 + \lambda_{\text{sc}} \left\| z \right\|_1, \qquad \lambda_{\text{sc}} > 0. \tag{5}$$

which finds a code z with few non-zero coefficients to reconstruct the input x. In other words, it finds the sparsest solution to the under-specified problem.

The classical solver to this problem (ISTA [4]) can be unrolled into a depth-L network of ISTA iterations, yielding learned ISTA (LISTA) [13]; in our setting, LISTA is an encoder yielding the sparse code $z = E_\theta(x)$, where the *support* is the set of active z coordinates. With a linear decoder, the support determines which decoder columns participate in reconstructing x, and the associated nonzero coefficients specify coordinates within that selected reconstruction subspace. When D and z are learned jointly, as in sparse coding [29], this induces a data-adaptive partition of the input state space where each region can be locally reconstructed with a code having the same support.

We therefore posit that combining Koopman representations and sparse coding objectives in a sparse Koopman autoencoder (SKAE) implicitly captures multibasin attractor dynamics without explicit basin labels in training. The Koopman objective encourages the continuous coefficients on each support to evolve linearly, while the sparsity objective encourages different local regimes to use different active coordinates. Thus, the model can represent a multibasin system using a discrete support pattern to identify the active basin and continuous coefficients for its localized Koopman (linear) representation. When ground-truth basin labels are available in benchmarks, we use them only after training to assess support–basin alignment or to construct diagnostic interventions.

3.2 Koopman autoencoder and sparse encoder

The predictor follows the KAE formulation in Section 2: an encoder, linear decoder, and latent transition, all of which are optimized jointly, produce the rollout in eq. (3). Like in sparse coding, [13] we constrain the columns of the decoder $D_\phi = D$ to be unit norm to avoid degenerate solutions. We experiment with K that is dense, block diagonal, or softly block-regularized.

Sparse encoding. Our main sparse encoder is LISTA [13]. It first computes a dense pre-code $c_t = f_\theta(x_t)$, then applies learned shrinkage refinements for $q = 0, \ldots, Q-1$:

$$u_t^{(0)} = \operatorname{shrink}(c_t, \tau), \qquad u_t^{(q+1)} = \operatorname{shrink}(S u_t^{(q)} + c_t, \tau), \qquad z_t = u_t^{(Q)}. \tag{6}$$

Here, $\operatorname{shrink}(a, \tau) = \operatorname{sign}(a) \max(|a| - \tau, 0)$ is applied element-wise, S is learned, and τ controls the threshold scale for shrinkage. The thresholding step creates exact zeros, so the encoder explicitly selects active latent coordinates while learning the selection rule from the prediction objective.

Transition families. We vary the structure of K separately from the encoder. The dense transition leaves K unrestricted. The block-diagonal transition partitions latent coordinates into M fixed groups and constrains

$$K_{\text{block}} = \operatorname{blkdiag}(K_1, \ldots, K_M). \tag{7}$$

We also use a soft-block LISTA ablation that keeps K dense but penalizes cross-group entries,

$$\mathcal{L}_{\text{offblock}} = \sum_{i,j:\, g(i) \neq g(j)} |K_{ij}|, \tag{8}$$

where $g(i)$ is the fixed architectural group containing coordinate i.

3.3 Training objective

Training uses adjacent observation windows. For a batch of B windows with prediction horizon L, the model produces reconstructed future states $x^{\mathrm{rec}}_{b,k+\ell}$, latent rollouts $z^{\mathrm{roll}}_{b,k+\ell}$, encoded future latents $z^{\mathrm{enc}}_{b,k+\ell}$, and decoded rollout predictions $\hat{x}_{b,k+\ell}$ as defined in eq. (4). The rollout is seeded by the initial state, and the following sums are over offsets $\ell = 1, \ldots, L$:

$$\bar{\mathcal{L}}_{\mathrm{pred}} = \frac{1}{BL\sqrt{d_x}} \sum_{b,\ell} \|\hat{x}_{b,k+\ell} - x_{b,k+\ell}\|_2 , \qquad \bar{\mathcal{L}}_{\mathrm{rec}} = \frac{1}{BL\sqrt{d_x}} \sum_{b,\ell} \left\|x^{\mathrm{rec}}_{b,k+\ell} - x_{b,k+\ell}\right\|_2 , \tag{9}$$

$$\bar{\mathcal{L}}_{\mathrm{lin}} = \frac{1}{BL} \sum_{b,\ell} \left\|z^{\mathrm{roll}}_{b,k+\ell} - z^{\mathrm{enc}}_{b,k+\ell}\right\|_2 , \qquad \bar{\mathcal{L}}_{\mathrm{sp}} = \frac{1}{BL} \sum_{b,\ell} \left\|z^{\mathrm{roll}}_{b,k+\ell}\right\|_1 . \tag{10}$$

The observation-space terms are normalized by $\sqrt{d_x}$ so that their scale is comparable across systems of different state dimension. The latent terms are left in their native scale because the latent dimension is fixed within each comparison.

The total objective is

$$\mathcal{L} = \frac{1}{L}\left(\lambda_{\mathrm{pred}}\bar{\mathcal{L}}_{\mathrm{pred}} + \lambda_{\mathrm{rec}}\bar{\mathcal{L}}_{\mathrm{rec}} + \lambda_{\mathrm{lin}}\bar{\mathcal{L}}_{\mathrm{lin}} + \lambda_{\mathrm{sp}}\bar{\mathcal{L}}_{\mathrm{sp}}\right) + \mathcal{L}_{\mathrm{struct}}. \tag{11}$$

Here $\mathcal{L}_{\mathrm{struct}} = 0$ for the dense and block-diagonal transition variants. For the soft-block transition, $\mathcal{L}_{\mathrm{struct}} = \lambda_{\mathrm{off}}\mathcal{L}_{\mathrm{offblock}}$, which probes whether a soft transition grouping is sufficient to induce useful support structure. The prediction and latent-linearity terms make the representation useful for Koopman rollout, the reconstruction term keeps the code tied to the observed state, and the sparsity term encourages selective activation.

3.4 Support objects

In this subsection, we introduce several notions of support that are used at evaluation time.

Let $z = E_\theta(x) \in \mathbb{R}^{d_z}$. A *support rule* first converts the latent code z into a Boolean active-coordinate vector $m(x) = \{|z_i| > 0\} \in \{0,1\}^{d_z}$, where $m_i(x) = 1$ means that latent coordinate i is active for state x. Equivalently, the exact support is the index set $S(x) = \{i : m_i(x) = 1\}$. Exact supports defined by an arbitrary threshold can be unstable: a small perturbation in x-space can lead to a different support in z, thus a single basin may be covered by several nearby or partially overlapping active-coordinate vectors. We therefore distinguish exact supports from *support families*, which group nearby Boolean support vectors and test whether these groups form basin-scale objects. Support families in general are more robust and lead to less basin fragmentation.

We use three support constructions:

- **Absolute-threshold support.** The absolute support vector is the Boolean mask denoted $m_{\mathrm{abs}}(x)$, where $m_{\mathrm{abs},i}(x) = \mathbf{1}\{|z_i| > 10^{-3}\}$ for indices i. $S_{\mathrm{abs}}(x) = \{i : m_{\mathrm{abs},i}(x) = 1\}$ is the equivalent active-index set. $m_{\mathrm{abs}}(x)$ is the input to support families such as F_{abs} (defined next), and to canonical wrong-support interventions.
- **Absolute-threshold support family.** F_{abs} is a group label for exact supports; every exact support m_{abs} is assigned an F_{abs} label. F_{abs} groups exact supports whose active-index sets substantially overlap. We define the overlap score as the Jaccard index, or intersection-over-union, between the active coordinates of two Boolean support vectors $m_{\mathrm{abs}}(x)$. On an evaluation collection $\mathcal{X}$, the algorithm first counts all distinct Boolean vectors and then visits those distinct vectors from most to least frequent, following a leader-style threshold clustering rule. Each family f stores one fixed representative vector r_f: the exact support vector that created that family. For the next distinct vector m, the algorithm computes the Jaccard index

$$J(m, r_f) = \frac{\sum_i \mathbf{1}\{m_i = 1 \text{ and } r_{f,i} = 1\}}{\sum_i \mathbf{1}\{m_i = 1 \text{ or } r_{f,i} = 1\}},$$

  with $J(0,0) = 1$ when both vectors are all zero. If the best existing representative has Jaccard similarity at least $\tau = 0.5$, every occurrence of m_{abs} receives that family label. Otherwise, m_{abs} starts a new family and becomes the representative r_f for that family. Pseudocode is provided in the appendix [TODO: appendix pseudocode]. We write $F_{\mathrm{abs}}(x)$ for the family assigned to $m_{\mathrm{abs}}(x)$.

3.5 Periodic re-encoding

If a support indicates the currently relevant local dynamics, a long rollout should be able to update that support. We therefore evaluate periodic re-encoding, following Fathi et al. [10]. Starting from $z_k = E_\theta(x_k)$ and a re-encoding period m, the model advances for m Koopman steps, decodes the predicted state, and encodes that prediction again:

$$\tilde{z}_{k+m} = K^m z_k, \qquad \tilde{x}_{k+m} = D_\phi(\tilde{z}_{k+m}), \qquad z_{k+m} = E_\theta(\tilde{x}_{k+m}). \tag{12}$$

The same procedure is repeated over the rollout horizon. Each rollout uses only the model's predicted state; it does not use the true future state, a basin label, or an oracle for transitions between basins. For each dynamical system, we tune a separate re-encoding period m on the training set.

4 Experiments

The experiments aim to evaluate if finite-dimensional Koopman representations can be learned for multi-attractor systems implicitly without resorting to side information such as basin labels. We first discuss the quality of learned Koopman representations from sparse KAEs via a forecasting-based evaluation. Next, in the context of sparse inference, we evaluate the importance of the specific active set of coordinates for forecasting within a basin of attraction. Finally, we evaluate alignment of support families F_{abs} with withheld basin labels.

Benchmark systems. The main benchmark for our experiments contains 15 procedurally generated two-dimensional multibasin systems. These systems are constructed by specifying potential wells around known attractor coordinates, and superimposing dynamics that encourage transitions to different far-away regimes; see Appendix **??**. Because we specify the locations of the potential wells, the basin labels are well-defined and used only for evaluation. We also use a subset of 10 chaotic systems from the Dysts repository [11, 12] with the step size dt multiplied by a factor of 30 as an additional forecasting test. Analytic expressions and more details are in Appendix **??**, **??**.

Training models. We compare six models shown in Table 1 by varying the encoder and latent transition structure and studying their effects in this setting. We test three LISTA sparse encoders (where LISTA-BD denotes block diagonal latent transitions and LISTA-SB denotes soft-block regularization), two sparse-latent MLP applying the sparsity penalty $\bar{\mathcal{L}}_{\text{sp}}$ (10) with dense [10] or block-diagonal (BD) transitions, and a dense-latent MLP control baseline with no sparsity incentive.

Most of these architectures are biased toward sparse inference by varying degrees. Our contribution is in studying how different ways of inducing sparsity can be useful. For LISTA, encoded states are sparse by construction via thresholding, and rollout latents get sparsity pressure from (10). For sparse MLPs, the encoded states have induced sparsity through the ReLU activations and ℓ_1 regularization [37, 23]. The dense MLP baseline has no intended zero-producing mechanism. Each model is trained for 15 random seeds on each system before evaluation. Training details are reported in Appendix **??**.

Statistical details. Point estimates are interquartile means (IQM) over seeds within each system to reduce seed uncertainty [2], then arithmetic means across systems. Horizon-trend lines use the same point estimate as the tables, with seed-bootstrap bands that summarize uncertainty over seeds for an average system. Statistical significance is rigorously tested with paired Wilcoxon signed-rank tests and Holm corrections; details are provided in Appendix **??**.

4.1 Forecasting experiments

Sparse encoders learn high-quality Koopman representations. The quality of the representation learned by a KAE is primarily measured by its ability to make accurate, multi-step forecasts. Thus, we evaluate the forecasting performance of trained sparse and dense-latent KAEs on the two benchmarks spanning 25 systems. Forecasting is evaluated on held-out trajectories at the stored observation cadence and summarized by mean squared error (MSE).

The results (Figure 2 and Table 1) show that sparse models yield a significant reduction of MSE forecasting error at all horizons on both sets of systems. In particular, at longer horizons, the dense MLP has 17-27x larger MSE at $H1000$ on the multibasin systems and about 1.5x larger MSE figs/fig_fixed15_horizon_curves.pdf

(a) 15-system multibasin benchmark.

figs/fig_dysts_dt30_forecasting_performance.pdf

(b) 10-system Dysts $dt{\times}30$ benchmark.

Figure 2: Forecasting horizon trends. Lines show arithmetic means across per-system seed-IQM MSEs on a log scale. Bands show fixed-system seed-bootstrap 95% intervals after system-wise log-relative normalization, anchored to the displayed mean, so they summarize seed uncertainty within an average system.

at $H4000$ on the Dysts systems compared to most of the sparse models (the LISTA-SB model outperforms the baseline but is noticeably worse than the rest). On the other hand, there are marginal differences between MSE predictions among the sparse-latent encoders. These results suggest that having encoded sparsity is crucial for good performance over all horizon prediction lengths.

Sparse supports are essential for good representations. If a support formed by sparse encoders is meant to carry basin-relevant dynamical information, then changing only the initial active set should damage forecasts, even when the coefficient values are otherwise kept fixed. We do an ablation study to show whether accurate forecasts within a basin of attraction require the correct coordinates to be active by isolating the selection of sparse coordinates from their values. We do this by forcing a sparse KAE to use an incorrect active set with specific interventions, and evaluating 20-step forecasting performance under these settings. The interventions are the following:

1. **Coordinate dropping:** Force the coordinates of z with the $k \in \{1, \ldots, 10\}$ largest magnitude coefficients to zero before forecasting.
2. **Random support:** the active coefficient values are preserved, but reassigned to randomly chosen inactive latent coordinates.

The results show that these interventions have a significant detrimental effect on forecast quality, even on very short horizons. At $H = 21$, the standard rollout has mean accumulated MSE 0.0158, compared to 0.508/1.37/2.08/3.26/8.51 for the rollouts after dropping the top 1/2/3/5/10 active

Table 1: Forecasting performance and compact basin-support diagnostics. Forecasting values are MSEs; lower is better. Each cell is an arithmetic mean across systems after summarizing seeds within each system by IQM. Superscripts mark Holm-corrected tests against the dense-latent MLP baseline: multibasin forecasting cells use within-system Wilcoxon/Holm tests, Dysts cells use exact sign tests across systems, and multibasin support-diagnostic cells use per-basin deep-slice Wilcoxon/Holm tests ($*$, $p < 0.05$). **Bold** marks the best entry in each directional column.

	Forecasting MSE↓						Support diagnostics	
	Multibasin, 15 systems			Dysts $dt{\times}30$, 10 systems			Multibasin, 15 systems	
Model	H100	H500	H1000	H100	H2000	H4000	$H(B \mid F_{\mathrm{abs}}) \downarrow$	$\lvert F_{\mathrm{abs}} \rvert$
LISTA	0.0407^*	0.144^*	0.166^*	$2.53{\times}10^{-4*}$	$\mathbf{0.0972}^*$	$\mathbf{0.452}^*$	$\mathbf{0.130}^*$	$\mathbf{4.0}$
LISTA-BD	0.0411^*	0.118^*	0.135^*	$3.38{\times}10^{-4}$	0.117^*	0.485^*	0.156^*	3.8
LISTA-SB	$\mathbf{0.0387}^*$	0.114^*	0.130^*	$4.85{\times}10^{-4}$	0.165	0.686	0.153^*	3.8
Sparse MLP, BD	0.0397^*	0.102^*	0.117^*	$2.33{\times}10^{-4*}$	0.119^*	0.493^*	0.257^*	3.3
Sparse MLP [10]	0.0397^*	$\mathbf{0.0940}^*$	$\mathbf{0.107}^*$	$\mathbf{2.32{\times}10^{-4*}}$	0.120^*	0.497^*	0.223^*	3.3
Dense MLP [25] *[baseline]*	0.830	2.68	2.93	0.00110	0.224	0.754	1.28	1.0

figs/fig_support_coordinate_dropping_accumulated_mse.pdf

(a) Dropping the largest active coordinates.

figs/fig_support_coordinate_random_shuffle_accumulated_m

(b) Moving active values to inactive coordinates.

Figure 3: Support-coordinate interventions on a multibasin system; interventions severely increase forecasting errors compared to the standard rollout. **(a)** The standard rollout is compared with cumulative dropping of the largest active coordinates of the initial sparse code; shaded bands are bootstrap 95% intervals over 100 initial conditions. **(b)** Active coefficient values are preserved but reassigned to randomly chosen inactive coordinates; the band is the interquartile range over shuffles.

Table 2: Accumulated MSE at $H = 21$ for the support interventions.

Rollout	Mean $\pm$ SD	Median [IQR]
Standard	0.0158 ± 0.0127	$0.0140\,[0.0055, 0.0224]$
Drop top-1	0.508 ± 0.647	$0.218\,[0.148, 0.677]$
Drop top-2	1.37 ± 0.938	$1.06\,[0.802, 1.63]$
Drop top-3	2.08 ± 1.10	$1.80\,[1.34, 2.10]$
Drop top-5	3.26 ± 2.44	$1.95\,[1.61, 4.17]$
Drop top-10	8.51 ± 6.83	$5.16\,[4.58, 8.11]$
Random support	$1.69 \times 10^3 \pm 2.19 \times 10^3$	$874\,[377, 2.06 \times 10^3]$

coordinates, and random support shuffling is much more destructive as seen in Figure 3. The trajectory panel (Figure 4a) shows the qualitative effects of the intervention. So, supports formed by sparse encoders do not merely correlate with basin labels; the active coordinates are key to the quality of the Koopman representation.

4.2 Support-basin alignment experiments

This mechanistic experiment examines the extent to which sparse support families fit after training can behave like basin-interior regime variables. States near separatrices are challenging and may confound the inferences: near these states, support changes can be induced by small perturbations in x, making supports unreliable for basin identification at separatrices. We therefore evaluate this diagnostic on held-out states deep in basins far from basin boundaries.

For a state x, let $d_1(x)$ and $d_2(x)$ be its distances to the nearest and second-nearest benchmark attractor centers. The *basin-depth margin* is $d_2(x) - d_1(x)$. This support-basin alignment experiment takes the top quartile of held-out states ranked by basin-depth margin within each benchmark basin, using ground-truth benchmark geometry to identify each basin. If a support family F_{abs} identifies a basin-specific regime used by the model, then on states deep inside a basin, it should leave little uncertainty about the withheld benchmark basin label B, measured by conditional entropy $H(B \mid F_{\text{abs}})$ as the main evaluation metric. $H(B \mid F_{\text{abs}})$ should be small when support families are basin-aligned. Also, different basins should be adequately represented by distinct support families, so we count the number of distinct support families formed in a system, denoted as $|F_{\text{abs}}|$, and compare it with the number of basins represented.

Support families F_{abs} identify basin membership. As seen in Table 1 and Figure 4b, LISTA F_{abs} support families provide more information to identify a ground truth basin label than any other model architecture, as seen by the lowest conditional entropy of 0.130. Other LISTA models with structured latent transitions are close competitors, followed by the sparse-latent MLPs, and then finally the dense-latent MLP baseline. LISTA-based models also expose the most support families, which better figs/fig_support_coordinate_trajectories_random_support_19.pdf

(a) Randomly shuffled support trajectories (red) compared to standard rollouts (black). Randomly shuffling the active coordinates leads to divergences.

figs/fig_fixed15_entropy_strips.pdf

(b) Distribution of conditional entropy results; Dense MLP cannot identify basins based on support families. Dots show system–seed pairs, gray I-bars mark first-to-third quartile ranges, and black bars mark IQM summaries for $H(B \mid F_{\mathrm{abs}})$.

matches the benchmark's basin-count mean/median 4.20/4; ideally, if a basin can truly be recovered by a single support family cluster, then the basin count should be one-to-one with the number of support families, $\overline{|F_{\mathrm{abs}}|}$. In stark contrast, the dense-latent MLP collapses the representation to one support family, so distinct basins have correlated learned latent dynamics. This suggests that the support families induced by sparse coding may serve as a good proxy for ground-truth basin labels; the sparse LISTA models may have the emergent ability to discover basins unsupervised, despite this not being explicit in the training objectives.

5 Discussion

This paper shows that sparse Koopman autoencoders can learn useful local regime structure in multibasin dynamical systems without observing basin labels. Existing theory and empirical evidence indicate that multibasin systems generally should not be forced through one continuous finite-dimensional global Koopman embedding [22, 5, 31, 10, 24]. Our results show that sparse codes offer a practical alternative: a single trained model *can* find good representations for multiple local Koopman regimes by using different latent supports. This gives practitioners a simple diagnostic object to inspect after training.

We qualify some limitations. First, our interpretability claims are evaluated on benchmark systems where basin geometry is known, and the support–basin alignment analysis is intentionally restricted to states deep inside basins. Near separatrices, small state changes can imply very different long-term outcomes, and support assignments become fragmented or unstable. This is not a failure mode unique to sparse Koopman models, but it means that support families should be interpreted as reliable basin-interior regime variables rather than perfect global basin classifiers. Second, while the experiments include a diverse set of synthetic multibasin systems and external chaotic flows, they remain deterministic benchmarks. Real-world systems may contain observation noise, partial observability, nonstationarity, control inputs, higher-dimensional attractors, or regime changes that are not cleanly basin-like.

More broadly, this work argues that interpretability in dynamical representation learning can come from the structure of the latent code. Sparse supports provide an interface between continuous Koopman coordinates and discrete regime structure, making them a promising tool for multi-regime forecasting, analysis, and eventually control. By showing that these supports improve prediction, are necessary for accurate rollouts, and align with withheld basin information, this paper positions sparse latent structure as a practical route toward Koopman models for multibasin dynamics.

References

[1] Aviad Aberdam, Alona Golts, and Michael Elad. Ada-LISTA: Learned Solvers Adaptive to Varying Models. *IEEE transactions on pattern analysis and machine intelligence*, 44(12): 9222–9235, December 2022. ISSN 1939-3539. doi: 10.1109/TPAMI.2021.3125041.

[2] Rishabh Agarwal, Max Schwarzer, Pablo Samuel Castro, Aaron C Courville, and Marc Bellemare. Deep reinforcement learning at the edge of the statistical precipice. *Advances in Neural Information Processing Systems*, 34, 2021.

[3] Omri Azencot, N. Benjamin Erichson, Vanessa Lin, and Michael Mahoney. Forecasting Sequential Data Using Consistent Koopman Autoencoders. In *Proceedings of the 37th International Conference on Machine Learning*, pages 475–485. PMLR, November 2020.

[4] Amir Beck and Marc Teboulle. A fast iterative shrinkage-thresholding algorithm for linear inverse problems. *SIAM journal on imaging sciences*, 2(1):183–202, 2009.

[5] Steven L. Brunton, Bingni W. Brunton, Joshua L. Proctor, and J. Nathan Kutz. Koopman Invariant Subspaces and Finite Linear Representations of Nonlinear Dynamical Systems for Control. *PLOS ONE*, 11(2), February 2016. ISSN 1932-6203. doi: 10.1371/journal.pone.0150171. URL `https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0150171`.

[6] Steven L. Brunton, Marko Budišić, Eurika Kaiser, and J. Nathan Kutz. Modern Koopman Theory for Dynamical Systems. *SIAM Review*, 64(2):229–340, May 2022. ISSN 0036-1445. doi: 10.1137/21M1401243. URL `https://doi.org/10.1137/21M1401243`.

[7] Scott Shaobing Chen, David L. Donoho, and Michael A. Saunders. Atomic Decomposition by Basis Pursuit. *SIAM Journal on Scientific Computing*, 20(1):33–61, 1998. doi: 10.1137/S1064827596304010. URL `https://doi.org/10.1137/S1064827596304010`.

[8] Xiaohan Chen, Jialin Liu, Zhangyang Wang, and Wotao Yin. Hyperparameter Tuning is All You Need for LISTA. In *Advances in Neural Information Processing Systems*, volume 34, pages 11678–11689. Curran Associates, Inc., 2021.

[9] David L. Donoho and Michael Elad. Optimally sparse representation in general (nonorthogonal) dictionaries via 1 minimization. *Proceedings of the National Academy of Sciences*, 100(5): 2197–2202, March 2003. doi: 10.1073/pnas.0437847100. URL `https://doi.org/10.1073/pnas.0437847100`.

[10] Mahan Fathi, Clement Gehring, Jonathan Pilault, David Kanaa, Pierre-Luc Bacon, and Ross Goroshin. Course correcting koopman representations. In *The Twelfth International Conference on Learning Representations*, 2024. URL `https://openreview.net/forum?id=A18gWgc5mi`.

[11] William Gilpin. Chaos as an interpretable benchmark for forecasting and data-driven modelling. In *Advances in Neural Information Processing Systems (NeurIPS) 2021*, 2021. URL `http://arxiv.org/abs/2110.05266`.

[12] William Gilpin. Model scale versus domain knowledge in statistical forecasting of chaotic systems. *Physical Review Research*, 5(4):043252, December 2023. doi: 10.1103/PhysRevResearch.5.043252. URL `https://link.aps.org/doi/10.1103/PhysRevResearch.5.043252`.

[13] Karol Gregor and Yann LeCun. Learning Fast Approximations of Sparse Coding. In *Proceedings of the 27th International Conference on Machine Learning*, ICML'10, pages 399–406, Haifa, Israel, 2010. Omnipress. ISBN 978-1-60558-907-7. doi: 10.5555/3104322.3104374. URL `https://dl.acm.org/doi/abs/10.5555/3104322.3104374`.

[14] D. M. Grobman. Homeomorphism of systems of differential equations. *Doklady Akademii Nauk SSSR*, 128:880–881, 1959.

[15] Lars Grüne and Jürgen Pannek. *Nonlinear Model Predictive Control*. Communications and Control Engineering. Springer International Publishing, Cham, 2017. ISBN 978-3-319-46023-9 978-3-319-46024-6. doi: 10.1007/978-3-319-46024-6. URL `http://link.springer.com/10.1007/978-3-319-46024-6`.

[16] Philip Hartman. A lemma in the theory of structural stability of differential equations. *Proceedings of the American Mathematical Society*, 11(4):610–620, 1960. doi: 10.1090/S0002-9939-1960-0121542-7.

[17] R.E. Kalman. On the general theory of control systems. *1st International IFAC Congress on Automatic and Remote Control, Moscow, USSR, 1960*, 1(1):491–502, August 1960. ISSN 1474-6670. doi: 10.1016/S1474-6670(17)70094-8. URL https://www.sciencedirect.com/science/article/pii/S1474667017700948.

[18] B. O. Koopman. Hamiltonian Systems and Transformation in Hilbert Space. *Proceedings of the National Academy of Sciences*, 17(5):315–318, May 1931. doi: 10.1073/pnas.17.5.315. URL https://www.pnas.org/doi/10.1073/pnas.17.5.315.

[19] B. O. Koopman and J. v. Neumann. Dynamical Systems of Continuous Spectra. *Proceedings of the National Academy of Sciences*, 18(3):255–263, March 1932. doi: 10.1073/pnas.18.3.255. URL https://www.pnas.org/doi/abs/10.1073/pnas.18.3.255.

[20] Milan Korda and Igor Mezić. Linear predictors for nonlinear dynamical systems: Koopman operator meets model predictive control. *Automatica*, 93:149–160, July 2018. ISSN 00051098. doi: 10.1016/j.automatica.2018.03.046. URL http://arxiv.org/abs/1611.03537.

[21] Matthew D. Kvalheim and Shai Revzen. Existence and uniqueness of global Koopman eigenfunctions for stable fixed points and periodic orbits. *Physica D: Nonlinear Phenomena*, 425:132959, November 2021. ISSN 0167-2789. doi: 10.1016/j.physd.2021.132959. URL https://www.sciencedirect.com/science/article/pii/S0167278921001160.

[22] Yueheng Lan and Igor Mezić. Linearization in the large of nonlinear systems and Koopman operator spectrum. *Physica D: Nonlinear Phenomena*, 242(1):42–53, January 2013. ISSN 0167-2789. doi: 10.1016/j.physd.2012.08.017.

[23] Oliver W. Layton, Siyuan Peng, and Scott T. Steinmetz. Relu, sparseness, and the encoding of optic flow in neural networks. *Sensors*, 24(23), 2024. ISSN 1424-8220. doi: 10.3390/s24237453.

[24] Zexiang Liu, Necmiye Ozay, and Eduardo D. Sontag. Properties of immersions for systems with multiple limit sets with implications to learning Koopman embeddings. *Automatica*, 176:112226, June 2025. ISSN 0005-1098. doi: 10.1016/j.automatica.2025.112226. URL https://www.sciencedirect.com/science/article/pii/S0005109825001189.

[25] Bethany Lusch, J. Nathan Kutz, and Steven L. Brunton. Deep learning for universal linear embeddings of nonlinear dynamics. *Nature Communications*, 9(1):4950, November 2018. ISSN 2041-1723. doi: 10.1038/s41467-018-07210-0.

[26] Giorgos Mamakoukas, Maria Castano, Xiaobo Tan, and Todd Murphey. Local Koopman Operators for Data-Driven Control of Robotic Systems. In *Robotics: Science and Systems XV*. Robotics: Science and Systems Foundation, June 2019. ISBN 978-0-9923747-5-4. doi: 10.15607/RSS.2019.XV.054. URL http://www.roboticsproceedings.org/rss15/p54.pdf.

[27] D. Q. Mayne, J. B. Rawlings, C. V. Rao, and P. O. M. Scokaert. Constrained model predictive control: Stability and optimality. *Automatica*, 36(6):789–814, June 2000. ISSN 0005-1098. doi: 10.1016/S0005-1098(99)00214-9. URL https://www.sciencedirect.com/science/article/pii/S0005109899002149.

[28] Igor Mezić. Spectral properties of dynamical systems, model reduction and decompositions. *Nonlinear Dynamics*, 41(1–3):309–325, 2005. doi: 10.1007/s11071-005-2824-x.

[29] Bruno A. Olshausen and David J. Field. Emergence of simple-cell receptive field properties by learning a sparse code for natural images. *Nature*, 381(6583):607–609, June 1996. ISSN 1476-4687. doi: 10.1038/381607a0. URL https://doi.org/10.1038/381607a0.

[30] Samuel E. Otto and Clarence W. Rowley. Linearly recurrent autoencoder networks for learning dynamics. *SIAM Journal on Applied Dynamical Systems*, 18(1):558–593, 2019. doi: 10.1137/18M1177846.

[31] Shaowu Pan and Karthik Duraisamy. On the lifting and reconstruction of nonlinear systems with multiple invariant sets. *Nonlinear Dynamics*, 112(12):10157–10165, June 2024. ISSN 1573-269X. doi: 10.1007/s11071-024-09581-0. URL https://doi.org/10.1007/s11071-024-09581-0.

[32] Peter J. Schmid. Dynamic mode decomposition of numerical and experimental data. *Journal of Fluid Mechanics*, 656:5–28, 2010. ISSN 0022-1120. doi: 10.1017/S0022112010001217.

[33] Haojie Shi and Max Q.-H. Meng. Deep Koopman Operator With Control for Nonlinear Systems. *IEEE Robotics and Automation Letters*, 7(3):7700–7707, July 2022. ISSN 2377-3766. doi: 10.1109/LRA.2022.3184036.

[34] Naoya Takeishi, Yoshinobu Kawahara, and Takehisa Yairi. Learning koopman invariant subspaces for dynamic mode decomposition. In *Advances in Neural Information Processing Systems 30*, volume 30, 2017. URL https://papers.nips.cc/paper/6713-learning-koopman-invariant-subspaces-for-dynamic-mode-decomposition.

[35] Matthew O. Williams, Ioannis G. Kevrekidis, and Clarence W. Rowley. A Data–Driven Approximation of the Koopman Operator: Extending Dynamic Mode Decomposition. *Journal of Nonlinear Science*, 25(6):1307–1346, December 2015. ISSN 1432-1467. doi: 10.1007/s00332-015-9258-5. URL https://doi.org/10.1007/s00332-015-9258-5.

[36] Matthew O. Williams, Clarence W. Rowley, and Ioannis G. Kevrekidis. A kernel-based method for data-driven koopman spectral analysis. *Journal of Computational Dynamics*, 2(2):247–265, May 2016. ISSN 2158-2491. doi: 10.3934/jcd.2015005. URL https://www.aimsciences.org/article/doi/10.3934/jcd.2015005.

[37] Yuchen Zhang, Jason D. Lee, and Michael I. Jordan. L1-regularized neural networks are improperly learnable in polynomial time. In Maria Florina Balcan and Kilian Q. Weinberger, editors, *Proceedings of The 33rd International Conference on Machine Learning*, volume 48 of *Proceedings of Machine Learning Research*, pages 993–1001, New York, New York, USA, 20–22 Jun 2016. PMLR. URL https://proceedings.mlr.press/v48/zhangd16.html.

NeurIPS Paper Checklist

The checklist is designed to encourage best practices for responsible machine learning research, addressing issues of reproducibility, transparency, research ethics, and societal impact. Do not remove the checklist: **The papers not including the checklist will be desk rejected.** The checklist should follow the references and follow the (optional) supplemental material. The checklist does NOT count towards the page limit.

Please read the checklist guidelines carefully for information on how to answer these questions. For each question in the checklist:

- You should answer [Yes], [No], or [N/A].
- [N/A] means either that the question is Not Applicable for that particular paper or the relevant information is Not Available.
- Please provide a short (1–2 sentence) justification right after your answer (even for [N/A]).

The checklist answers are an integral part of your paper submission. They are visible to the reviewers, area chairs, senior area chairs, and ethics reviewers. You will also be asked to include it (after eventual revisions) with the final version of your paper, and its final version will be published with the paper.

The reviewers of your paper will be asked to use the checklist as one of the factors in their evaluation. While [Yes] is generally preferable to [No], it is perfectly acceptable to answer [No] provided a proper justification is given (e.g., error bars are not reported because it would be too computationally expensive" or "we were unable to find the license for the dataset we used"). In general, answering [No] or [N/A] is not grounds for rejection. While the questions are phrased in a binary way, we acknowledge that the true answer is often more nuanced, so please just use your best judgment and write a justification to elaborate. All supporting evidence can appear either in the main paper or the supplemental material, provided in appendix. If you answer [Yes] to a question, in the justification please point to the section(s) where related material for the question can be found.

IMPORTANT, please:

- **Delete this instruction block, but keep the section heading "NeurIPS Paper Checklist"**,
- **Keep the checklist subsection headings, questions/answers and guidelines below.**
- **Do not modify the questions and only use the provided macros for your answers**.

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: We have checked this and feel that our claims reflect the paper's contribution and scope.

   Guidelines:

   - The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

Justification: Section 5 explicitly identifies what the experiments do and do not establish.

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate "Limitations" section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

   Answer: [N/A]

   Justification: No original theorems or proofs or lemmas were included in this work.

   Guidelines:

   - The answer [N/A] means that the paper does not include theoretical results.
   - All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
   - All assumptions should be clearly stated or referenced in the statement of any theorems.
   - The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
   - Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
   - Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

   Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

   Answer: [Yes]

   Justification: Section 4 and Appendixes **??**, **??**, **??** contains benchmark populations, model families, training budgets, evaluation horizons, statistical units, and seed aggregation rules.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.

- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
  (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
  (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
  (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
  (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

   Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

   Answer: **[TODO]**

   Justification: **[TODO]**

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
   - The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
   - The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
   - The assumptions made should be given (e.g., Normally distributed errors).
   - It should be clear whether the error bar is the standard deviation or the standard error of the mean.
   - It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
   - For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
   - If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

   Answer: **[TODO]**

   Justification: **[TODO]**

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
   - The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
   - The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

   Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

   Answer: **[TODO]**

   Justification: **[TODO]**

   Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

    Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

    Answer: **[TODO]**

    Justification: **[TODO]**

    Guidelines:

    - The answer [N/A] means that there is no societal impact of the work performed.
    - If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
    - Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
    - The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
    - The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
    - If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

    Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

    Answer: **[TODO]**

    Justification: **[TODO]**

    Guidelines:

    - The answer [N/A] means that the paper poses no such risks.
    - Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
    - Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
    - We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

    Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

    Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

    Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

    Answer: **[TODO]**

    Justification: **[TODO]**

    Guidelines:

    - The answer [N/A] means that the paper does not release new assets.
    - Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
    - The paper should discuss whether and how consent was obtained from people whose asset is used.
    - At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

    Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

    Answer: **[TODO]**

    Justification: **[TODO]**

    Guidelines:

    - The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
    - Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
    - According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

    Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

    Answer: **[TODO]**

    Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

    Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

    Answer: **[TODO]**

    Justification: **[TODO]**

    Guidelines:

    - The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
    - Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.